EIP Support Guide

Technical Track

Tech 5

2025/2026

Introduction

Welcome to the Technical track of your Epitech Innovative Project (EIP) in Tech 5 – your final year to demonstrate true engineering maturity.

This year is about technical excellence and reusable value. You've built your solution, explored technologies, and structured your architecture. Now it's time to elevate your work to professional standards. Your mission: create technical assets that have value beyond your project – components others can use, expertise others can learn from, and engineering practices that scale.

Tech 5 represents a shift from **building features** to **engineering solutions**. This year, you'll need to:

- **Industrialize your code** with the rigor of production-ready software
- **Master advanced technologies** through deep research and concrete application
- **Optimize ruthlessly** for performance, stability, and security with measurable results
- **Share your knowledge** like a senior engineer contributing to the tech community
- **Think beyond your project** to create reusable components, tools, or APIs

This guide provides you with a structured framework to achieve technical excellence. You'll work on **two key objectives**:

1. **One mandatory objective** (choose between two options) that will push you to either industrialize a technical component with professional standards OR deepen an advanced technological expertise with documented research and application.
2. **One complementary objective** (choose from four options) focused on technical impact – whether through performance optimization, knowledge sharing, creating developer tools, or getting expert validation.

Remember: by the end of this year, you should have tangible technical assets that demonstrate your engineering maturity. This means production-quality code, documented expertise, measurable improvements, and contributions that extend beyond your immediate project.

Your mentor is here as your technical advisor – leverage their experience to challenge your architectural decisions, validate your technical choices, and ensure you're building with professional standards. But ultimately, this is your opportunity to prove you're ready to be a senior engineer.

Mandatory Objectives (choose one)

1 - Industrialize a technical component of the project

Identify & isolate the component

- Choose a **clearly delimited functional or technical brick** with strong reuse potential.
  Examples: scoring engine, data parser, API wrapper, custom UI component, search engine, AI module, etc.
- Isolate this brick in a **dedicated folder** or **separate repository**, respecting modularization standards (SRP, separation of concerns, encapsulation, etc.).

Code quality hardening

- Refactor for **modularity, genericity, typing, and readability**.
- Implement **automated tests**: unit, behavioral, and/or performance.
- Integrate **quality gates/tools**: linter, formatter, static analysis, coverage.

Complete technical documentation

- Write a **professional README**: description, dependencies, install, configuration, usage examples, license.
- Add developer docs as needed (Notion, GitHub Wiki, Docusaurus, etc.).
- If relevant, create a **demo or playground** to illustrate usage.

Packaging & distribution

- Provide an access mode:
  as an **open-source or private package** (npm, PyPI, GitHub Packages, etc.),
  or as an **integrable brick** via API, SDK, or public repo.
- Structure a **distribution strategy**: usage in other projects, peer testing, external feedback.

Expected deliverables (non-exhaustive)

- Link to the **isolated brick** (repo or module).
- **Professional README** (goals, usage, examples, license).
- **Test files + coverage reports**.
- **Code quality report** (linter, static analysis, SonarQube score, etc.).
- **Integration examples** in other contexts (docs, screenshots, snippets).
- **Refactoring journal / technical trade-offs** recorded.

What is not sufficient

- Leaving the brick **buried** in the main codebase with no clear separation.
- A module that is **untested** or **undocumented**.
- Copying an existing component **without understanding** or adaptation.
- Publishing a repo with only an **empty README** or raw code dump.
- Producing a **private brick** with no diffusion plan or proof of reusability.

2 - Deepen an advanced technological expertise

Choose a strategic expertise topic

- Select an advanced topic related to the project, e.g.:
  Generative AI, LLMs, NLP, edge computing, distributed systems, advanced CI/CD, microservices, cybersecurity, privacy-by-design, data engineering, developer tooling, real-time databases, etc.
- Justify the choice by expected project impact and market relevance.

In-depth, structured research

- Conduct a **multi-source watch** (articles, docs, forums, videos, conferences, white papers, etc.).
- Compare **≥ 3 approaches/tools/architectures**.
- Build a **benchmark** with concrete criteria: performance, scalability, ecosystem, security, ergonomics, costs, etc.

Produce a clear, actionable synthesis

- Deliver a **structured technical dossier** readable by peers/mentors, including:
  topic map, *approach comparison*, *experiments or tests*, **clear technical recommendations**.

Apply learning inside the project

- Apply the expertise concretely to part of the project:
  architecture change; integration of a specialized tool/lib/framework; rethought approach; performance/scalability improvement.
- **Justify decisions** based on research.

External valorization of expertise

- Share all or part of the work as an **article, talk, or public/internal synthesis**.
  Optional: feedback from an external expert, publication on a tech platform, presentation to other teams.

Expected deliverables (non-exhaustive)

- **Expertise topic sheet** (stakes, choice, context).
- **Structured technical synthesis** (10–20 pages or equivalent visuals).
- **Comparative benchmark** across tools/approaches.
- **Demo or explanation** of concrete application in the project.
- **Slides or video** presenting the expertise to a non-expert technical audience.
- *Optional:* link to an external publication or expert feedback.

What is not sufficient

- Skimming articles **without** producing a synthesis or applying anything.
- Summarizing a technology **without** comparison or testing.
- Integrating a tool **"because everyone uses it"** without justification.
- Copying an approach **without** understanding alternatives.
- Failing to document **concrete impacts** on the project.

Complementary Objectives (1 to be chosen by the group, in consultation with the mentor)

A - Improve performance, stability, or security of the project

Target a critical technical axis

Choose a priority objective among:

- Improve **load/execution times**.
- Reduce **memory/CPU consumption**.
- Increase **load resilience** (concurrent users, data volume).
- **Secure** code or infrastructure (vulnerabilities, permissions, injection, tokens, etc.).
- Increase **resilience** to errors or unexpected behaviors.

Establish tracking metrics

- Define **2–3 measurable indicators** (latency, memory, error rate, response time, Lighthouse score, SAST/DAST results, etc.).
- Set up a **local or external dashboard** (Prometheus, Sentry, Grafana, Datadog, console logs, Chrome audit, etc.).
- Run an **initial quantified diagnosis**.

Tests, simulations, or audits

- Run one or more of:
  - **stress/load tests**, **resilience tests**;
  - **security audits** (automated/manual);
  - **static code analysis**;
  - **profilers**, **Lighthouse**, **end-to-end tests**, **failover** on critical scenarios.
- Identify **critical zones** to optimize or secure.

Optimization or hardening

- Implement improvements:
  - targeted **refactors**;
  - **cache**, **lazy load**, **CDN**, **batch/async**, **circuit breakers**, **retries**;
  - **security checks**, input validation, least-privilege, **hashing/encryption**.
- Compare **before/after** results; record trade-offs (gain vs. cost vs. complexity).

Expected deliverables (non-exhaustive)

- **Initial diagnosis sheet**: metrics, context, identified problem.
- **Audit/test report** (tools used, logs, results).
- **Dashboard or comparative report captures** (before/after).
- **Optimization changelog**: impacted code, modules changed, applied strategy.
- **Trade-off analysis**: choices made, rejected, side effects.
- **Impact synthesis** on project stability or scalability.

What is not sufficient

- Claiming the project “runs fine” **without** measurement or proof.
- **Intuition-only** optimization, with no tests or tracked indicators.
- Only reducing an image or static file size.
- Installing a tool (e.g., Sentry) **without using it actively**.
- Adding tests **without** documenting results.

B - Publicly share technical knowledge

Define a relevant technical topic

- Choose a topic from the project/Technical Track with **interest potential**, e.g.:
  "How we built our analysis engine," "5 Docker mistakes we made," "Refactoring a Node microservice to TypeScript," "Benchmark of 3 AI APIs," "Real-time system rollout: lessons learned," etc.
- Ensure **sufficient mastery** and that the topic is **illustrable** with a concrete case or demo.

Choose format & platform

- Format examples:
  Written article (Medium, Dev.to, blog, LinkedIn),
  Technical video (YouTube, Loom, LinkedIn short, tutorial),
  Technical post/thread (LinkedIn, Twitter/X),
  Talk/presentation (recorded or live).
- The content should be **publicly shareable** (even at small scale) or within an identified community.

Produce & publish

- Structure content: **clear title, hook, outline, technical elements, illustrations, code snippets, conclusion**.
- Address **developers/makers/student engineers**.
- Add **personal style/experience** (errors, choices, results, regrets, anecdotes).

Engage & learn

- Publish on a platform with **engagement** (comments, views, likes, discussions).
- Track feedback: **shares, reactions, comments**.
- Iterate if needed (e.g., follow-up content, summary, talk version).

Expected deliverables (non-exhaustive)

- **Link to published content** (video, shared file, LinkedIn post, Dev.to article, etc.).
- **Script or outline** (doc or Notion).
- **Performance captures**: views, reactions, comments.
- **Engagement/feedback synthesis**.
- *Optional:* trace of a **live/oral presentation** (even internal).
- **Reuse** in pitch, README, portfolio, or CV.

What is not sufficient

- Posting a **bare GitHub link** with no explanation.
- Writing **too short/generic** content ("Our project is X…") with no pedagogy.
- Reposting internal docs **without** teaching intent.
- Creating an article **without publishing** it or without a target audience.
- Limiting to **self-promo** with no technical substance.

C - Create a mini-tool, SDK, or open API around the project

Identify an opportunity inside the project

- Spot a part of the project or secondary tech that can be **extracted**:
  format converter, external API wrapper, **JS SDK**, **deployment CLI**, internal monitoring tool, **RBAC module**, mini auth backend, **UI component library**, etc.
- Define a **generic use case** or recurring need that would benefit from this tool.

Structure the module or API

- Organize as a **clean, separate, modular repo**.
- Structure code to:
  expose a **simple interface** (REST endpoint, CLI method, SDK functions),
  decouple from the main project,
  integrate **tests** and **versioning**.

Clear documentation & fast onboarding

- Write a professional **README** with:
  module purpose, primary use case, install/config, example calls (snippets), available commands (for a CLI), constraints/limitations.
- *Optional:* build a **live demo/sandbox** (Postman, CodeSandbox, Swagger).

Publication & external usage

- Publish on the right platform:
  GitHub (open repo with clear license), **npm**, **PyPI**, **Packagist**, **Docker Hub**, etc.
- Test **integration in another project** (even simple) or with an external developer.
- Collect **feedback** on usability or value.

Expected deliverables (non-exhaustive)

- Link to the **repo/instance** (GitHub, npm, Docker, Swagger, etc.).
- **Complete README** + config files (package.json, setup.py, etc.).
- **Code samples** or integration tutorials.
- **Unit test captures** or proof of external usage.
- **External/internal user feedback** (verbatim, issue, pull request).
- *Optional:* **open-source badge** or listing in a tech directory.

What is not sufficient

- Extracting a file/folder **without docs** or use case.
- Creating an API **without documented routes** or clear logic.
- Missing **install/run/integration** instructions.
- Building a tool **too coupled** to the original project (not generalizable).
- Failing to **publish** or **test real usage**.

D - Participate in a technical review with an external expert

Identify an external expert

- Look for a qualified profile: **senior developer, software architect, OSS contributor, CTO, alumni, tech mentor, researcher**.
- Sourcing channels:
  - **GitHub** (creators/contributors of related projects),
  - **LinkedIn/Discord/Slack** in tech communities,
  - **Epitech networks**, alumni, partners,
  - personal contacts or via mentors.

Prepare the technical review

- Define topics to challenge:
  - architecture, technology choices, data model, business logic, patterns, scalability, performance, API design, etc.
- Prepare a **clear, concise review pack**:
  - code excerpt or repo link,
  - architecture diagram,
  - **specific questions**,
  - unresolved issues or uncertainties.

Run the exchange

- Organize a **30–60 min** synchronous session (video, in-person, Discord), or submit the pack for **written/video** feedback.
- **Document** the exchange: notes, verbatims, received advice, given answers.

Apply & capitalize

- Implement **at least one** improvement/modification from the exchange.
- Write a **technical memo**: what the team kept, contested (if any), and changed.
- **Valorize** the exchange: mention in docs, pitch, or final synthesis.

Expected deliverables (non-exhaustive)

- **Expert sheet**: name, profile, link to tech activity.
- **Prepared review pack**: code, questions, context.
- **Proof of exchange**: call capture, written feedback, testimony, email reply.
- **Synthesis memo**: advice received, decisions taken.
- **Applied modification** in code/architecture (commit, changelog, explanation).
- **Public acknowledgment** (optional but valuable).

What is not sufficient

- Discussing with a **classmate** or an **already involved** mentor only.
- Contacting an expert with **no clear objective** or preparation.
- Presenting a review with **no trace** or proof of the exchange.
- Refusing any adjustment after **reasoned feedback** with no justification.
- Settling for **generic opinions** (e.g., “looks cool”) with no technical questions.